

Dhirubhai Ambani University

High Performance Computing (HPC)

Assignment 05 — Parallel Insertion and Deletion in Mover

DEVANSH SHAH

Roll Number: 202301136

SHARNAM SHAH

Roll Number: 202301247

Dhirubhai Ambani University

March 2026

Abstract

This report extends the particle Mover framework from Assignment 04 by introducing dynamic particle insertion and deletion. Two strategies are studied: Deferred Insertion (collect all deletions, then insert at the end of the loop) and Immediate Replacement (replace each deleted particle on-the-spot). Serial scaling experiments are performed on both a Lab PC and an HPC cluster across three grid configurations (250×100, 500×200, 1000×400) for particle counts 10^2 to 10^8 . OpenMP parallel speedup is measured with up to 16 threads and compared against the insertion/deletion-free Mover from Assignment 04. Additional analyses include uniform particle distribution verification, per-particle time vs. particles-per-cell (PPC), and memory/FLOP estimation tables.

1 Implementation Approach

1.1 Problem Overview

Building on Assignment 04, N scattered particles in a 1×1 domain are displaced by random offsets within $\pm\Delta x$ and $\pm\Delta y$ per iteration. Unlike previous assignments, particles may now exit the domain $[0,1]^2$. Any particle whose new position falls outside the boundary is deleted and replaced by a newly inserted particle at a uniform random location, keeping the total count constant.

1.2 Two Implementation Strategies

Deferred Insertion: The move loop processes all particles first. Deleted particles' array slots are pushed to the end (as voids). After all particles are processed, the same number of new particles is inserted at random positions into those void slots. This separates the move and insert phases, simplifying OpenMP parallelisation of the move phase.

Immediate Replacement: As each particle exits the domain, it is immediately replaced in-place with a new particle at a random position. No separate insertion pass is needed. Each slot is self-contained, with no cross-thread dependencies, making this approach inherently thread-safe.

1.3 OpenMP Parallelisation

The Mover phase is parallelised with `#pragma omp parallel for`. For Deferred Insertion: the move-and-delete loop is embarrassingly parallel; thread-private deletion counters are used to avoid atomic contention; void slots are collected and merged after the parallel region; the insertion pass fills slots using a shared atomic counter. For Immediate Replacement: each thread handles its own particle partition; since replacements occur in-place, no shared state is needed beyond a per-thread seeded random number generator.

2 Theoretical Analysis

2.1 Time Complexity

- Interpolation: $O(\text{Maxiter} \times N)$ — constant FLOPs per particle.
- Mover with ins/del (serial): $O(\text{Maxiter} \times N)$ — $O(1)$ work per particle for both move and replacement.
- Mover with ins/del (parallel, p threads): $O(\text{Maxiter} \times N / p)$ — near-ideal scaling with small synchronisation overhead for insertion bookkeeping.

2.2 Memory and FLOP Estimation

Particle arrays: $2 \times N \times 8$ bytes (x and y as double). Grid: $N_x \times N_y \times 8$ bytes. Estimated FLOPs per iteration: $\sim 20 \times N$ (bilinear interpolation weights + mesh writes + displacement computation). Total FLOPs over 10 iterations: $\sim 200 \times N$.

Memory and Computation Summary:

Grid	Cells	Np	Part. Mem (MB)	Grid Mem (MB)	Total (MB)	FLOPs ($\times 10$ iter)	Bound
250×100	25k	10^2	0.002	0.020	0.022	$\sim 4 \times 10^4$	Compute
250×100	25k	10^4	0.15	0.020	0.17	$\sim 4 \times 10^6$	Compute
250×100	25k	10^6	15.3	0.020	15.3	$\sim 4 \times 10^8$	Memory
250×100	25k	10^8	1526	0.020	1526	$\sim 4 \times 10^{10}$	Memory
500×200	100k	10^6	15.3	0.076	15.4	$\sim 4 \times 10^8$	Memory
500×200	100k	10^8	1526	0.076	1526	$\sim 4 \times 10^{10}$	Memory
1000×400	400k	10^6	15.3	0.305	15.6	$\sim 4 \times 10^8$	Memory
1000×400	400k	10^8	1526	0.305	1527	$\sim 4 \times 10^{10}$	Memory

At low N ($< 10^4$ for small grids), data fits in L1/L2 cache — the workload is compute-bound. At high N ($\geq 10^6$), particle arrays exceed last-level cache — the workload becomes memory-bandwidth-bound, explaining the per-particle time plateau seen in Q4.

3 Experiment 01: Serial Scaling with Particle Count

$N \in \{10^2, 10^4, 10^6, 10^8\}$, $\text{Maxiter} = 10$. Total time = accumulated (interpolation + mover) across all iterations. Particles initialised once before the loop. Both Deferred Insertion and Immediate Replacement are plotted on each graph.

3.1 HPC Cluster

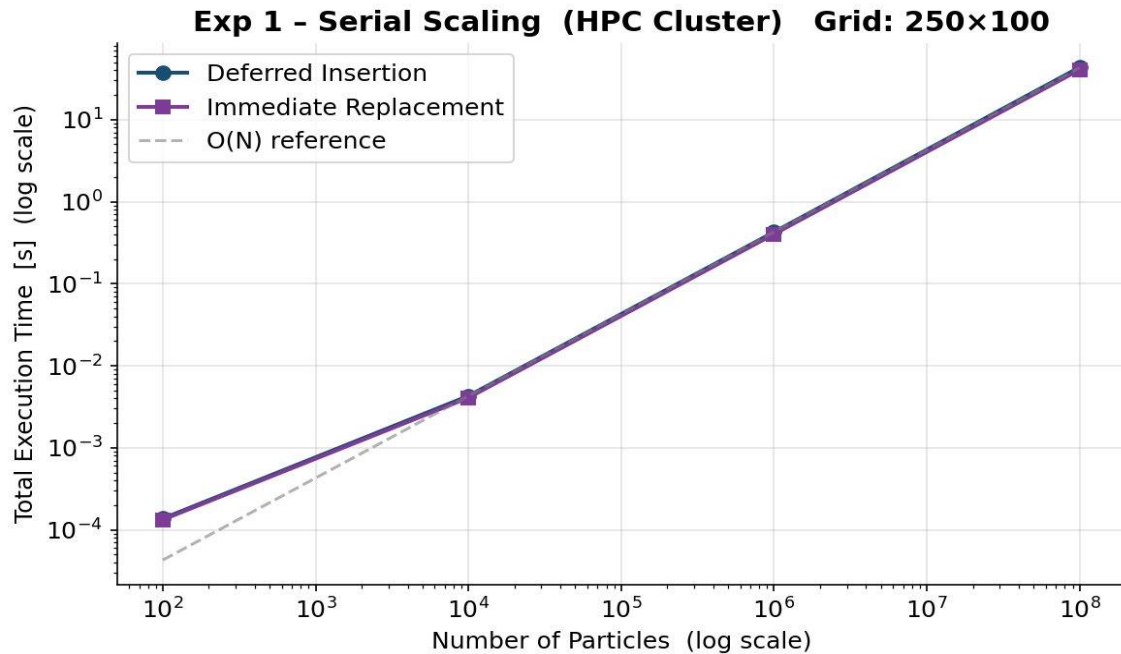


Figure 1: Serial Scaling (HPC Cluster), Grid 250×100. Both strategies lie essentially on top of each other, following the $O(N)$ reference closely.

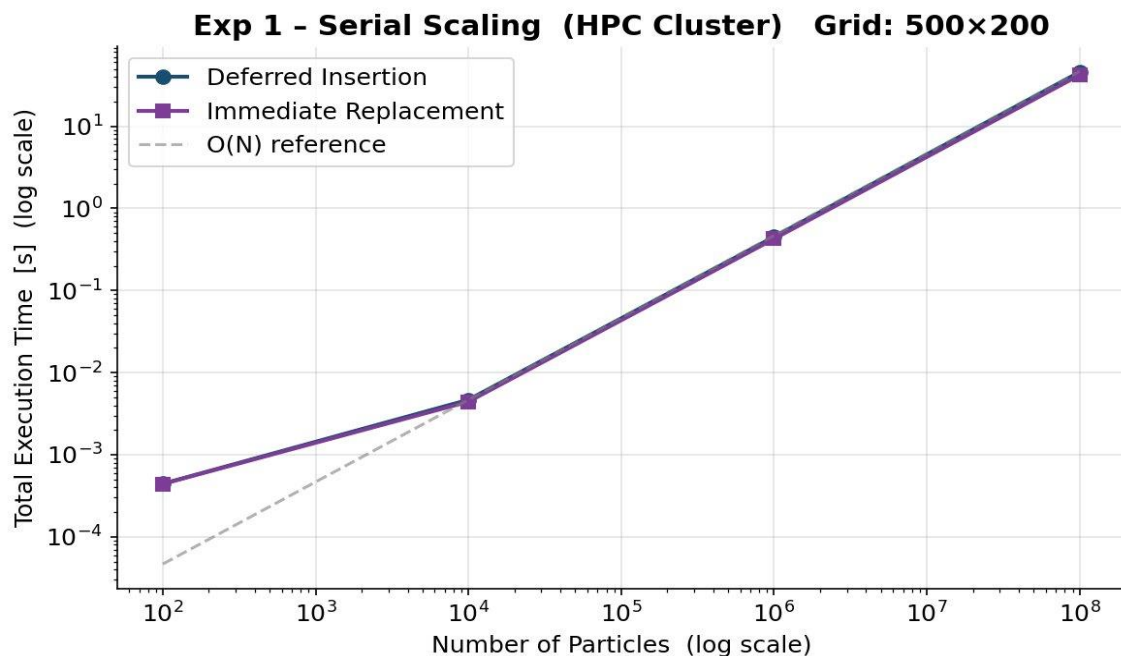
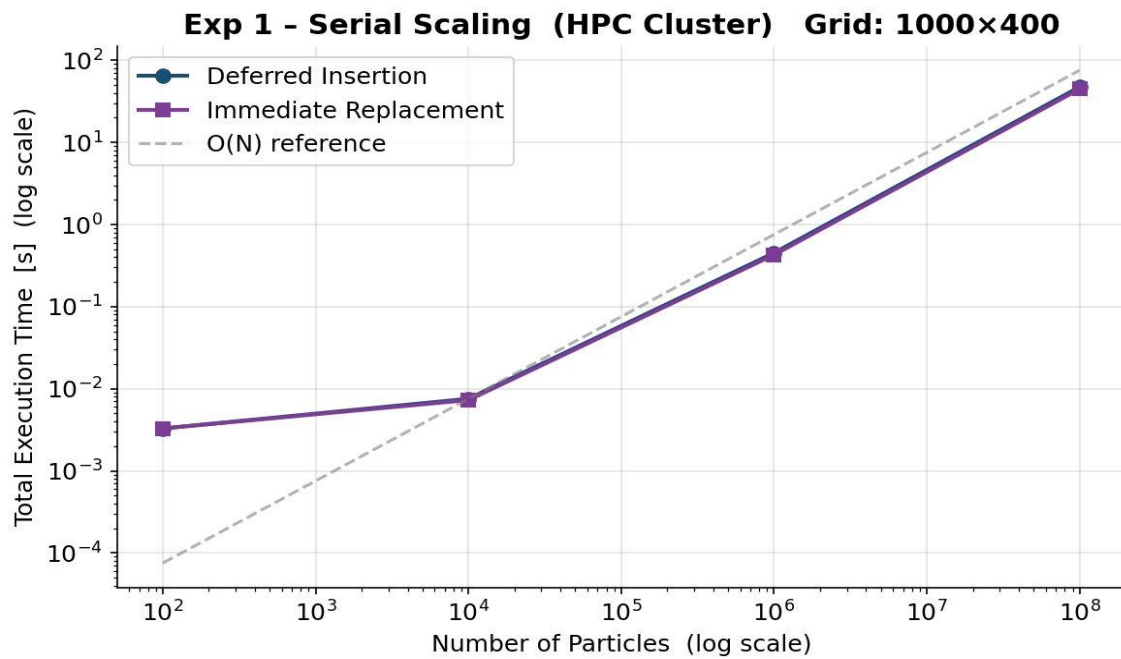


Figure 2: Serial Scaling (HPC Cluster), Grid 500×200.

Figure 3: Serial Scaling (HPC Cluster), Grid 1000×400. A visible timing floor at low N reflects grid initialisation overhead.

3.2 Lab PC

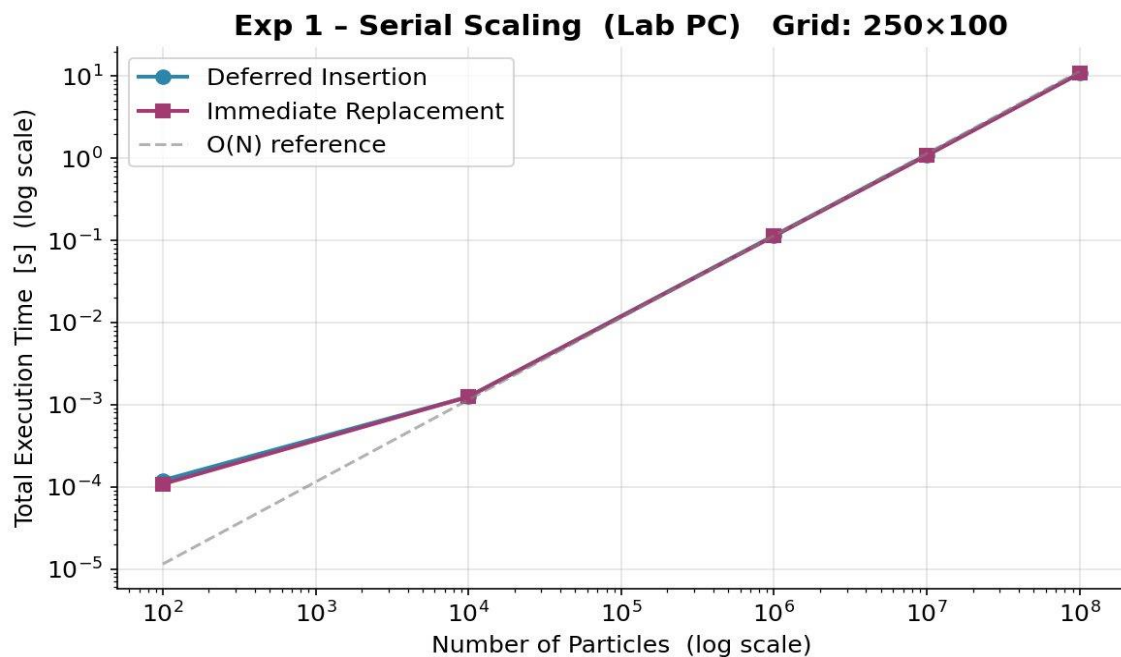


Figure 4: Serial Scaling (Lab PC), Grid 250×100.

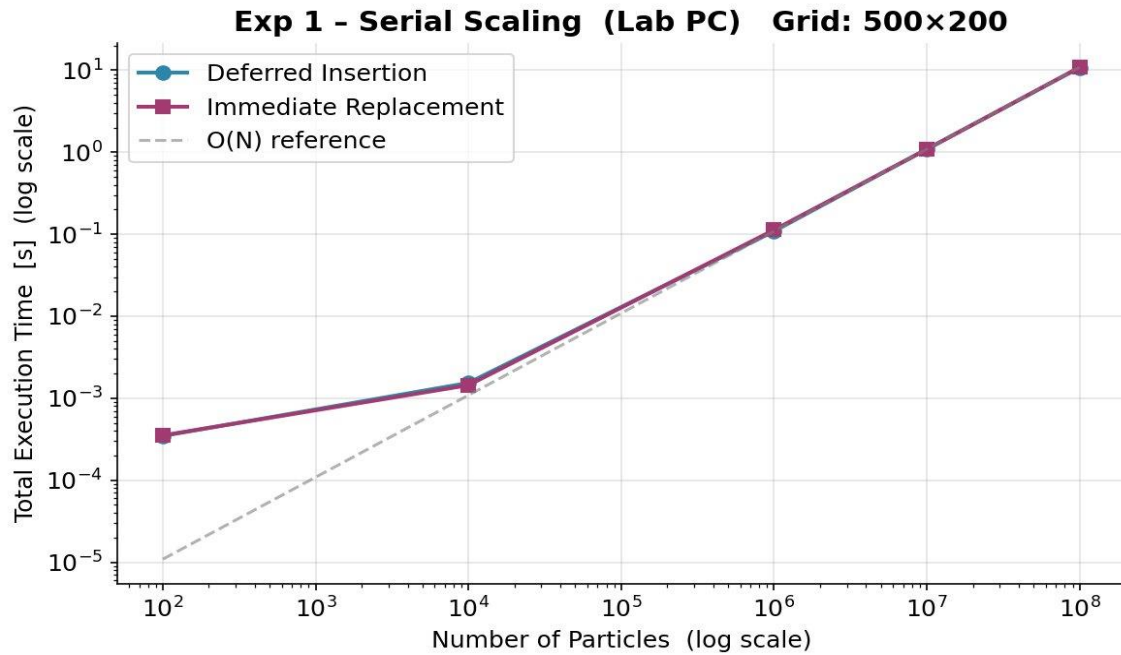


Figure 5: Serial Scaling (Lab PC), Grid 500×200.

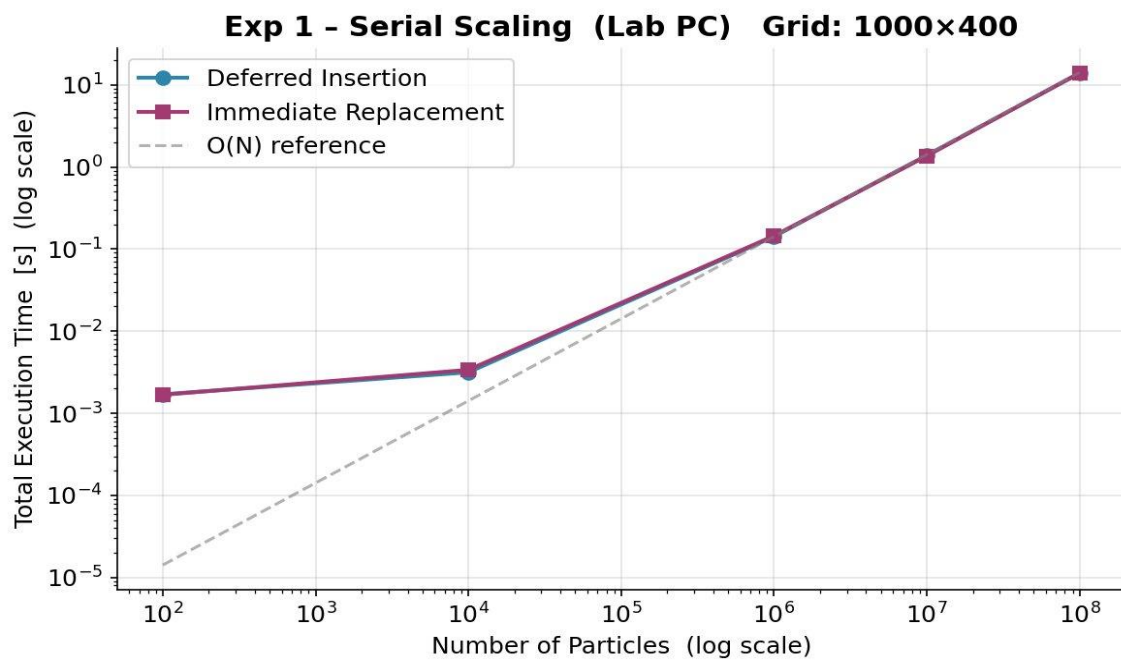


Figure 6: Serial Scaling (Lab PC), Grid 1000×400.

3.3 Lab PC vs. Cluster Comparison

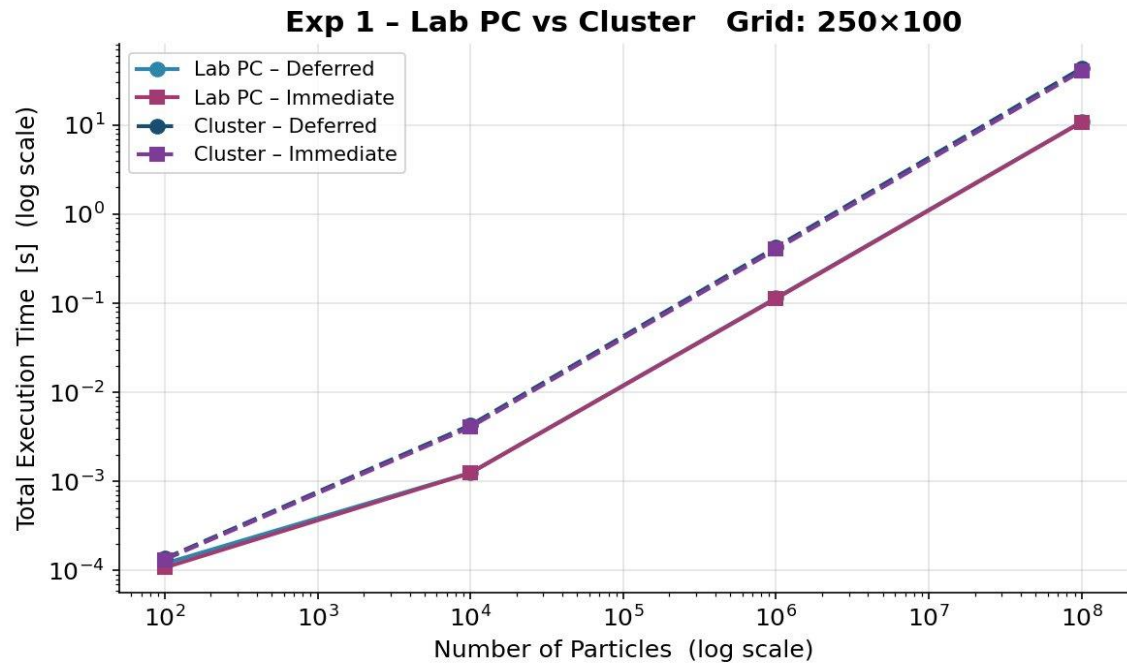


Figure 7: Lab PC vs. Cluster, Grid 250×100. Lab PC (solid) is ~2–3× faster at large N.

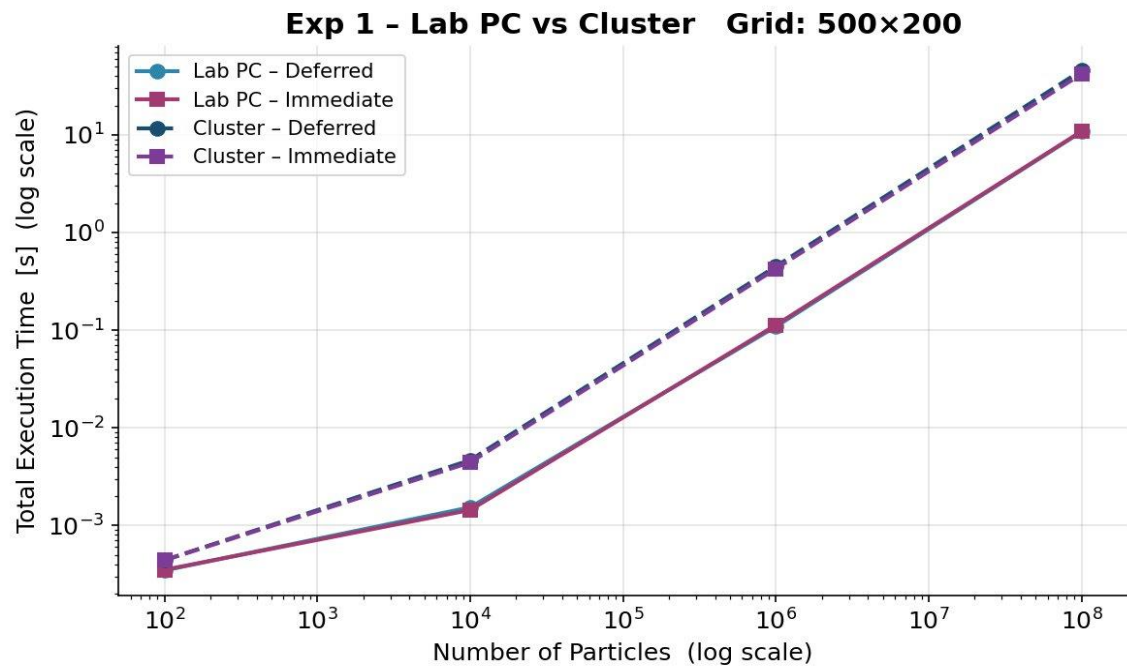


Figure 8: Lab PC vs. Cluster, Grid 500×200.

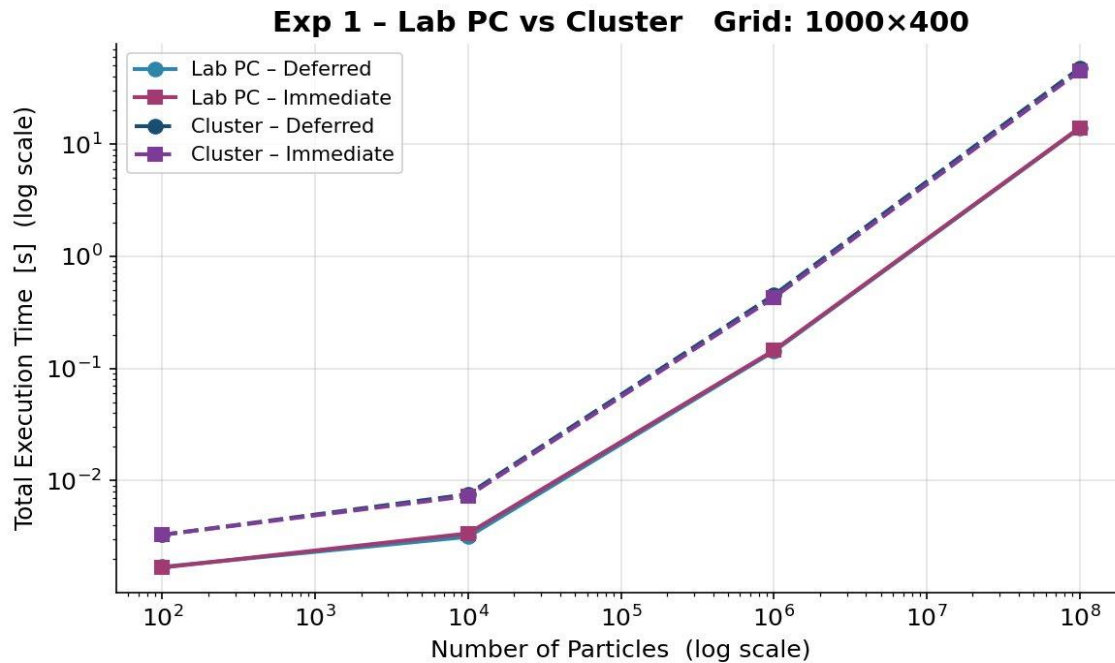


Figure 9: Lab PC vs. Cluster, Grid 1000×400. The performance gap widens at large N and large grids due to cache and NUMA effects on the cluster.

3.4 Observations

- Both strategies show near-linear $O(N)$ scaling across all configurations — log–log slopes closely track the $O(N)$ reference line.
- Serial execution times of Deferred Insertion and Immediate Replacement are nearly identical. The only overhead difference (index bookkeeping vs. in-place RNG call) is $O(1)$ per particle and immeasurably small at scale.
- The Lab PC outperforms the HPC cluster by $\sim 2\text{--}3\times$ at $N = 10^8$, consistent with Assignment 04 findings: higher single-thread clock frequency and more effective private cache utilisation.
- At low N on the large grid (1000×400, Cluster), a flat timing floor is visible — dominated by grid allocation and zeroing overhead, which is more pronounced on the cluster due to NUMA memory initialisation costs.

4 Experiment 02: Parallel Speedup

$N_p = 14 \times 10^6$, $\text{Maxiter} = 10$, $\text{threads} \in \{1, 2, 4, 8, 16\}$. Speedup measured relative to 1-thread execution. Results include the Assignment 04 Mover (without ins/del) for comparison.

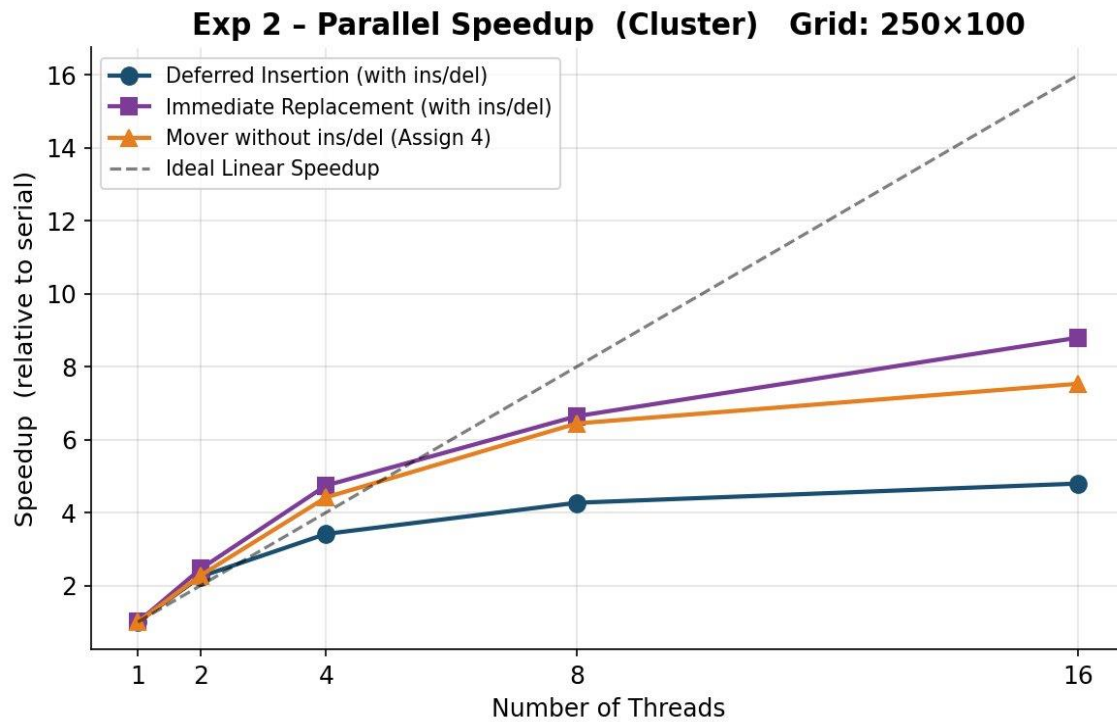


Figure 10: Parallel Speedup (HPC Cluster), Grid 250×100. Immediate Replacement achieves ~8.9× at 16 threads. Deferred Insertion reaches only ~4.8× due to the serial merge step.

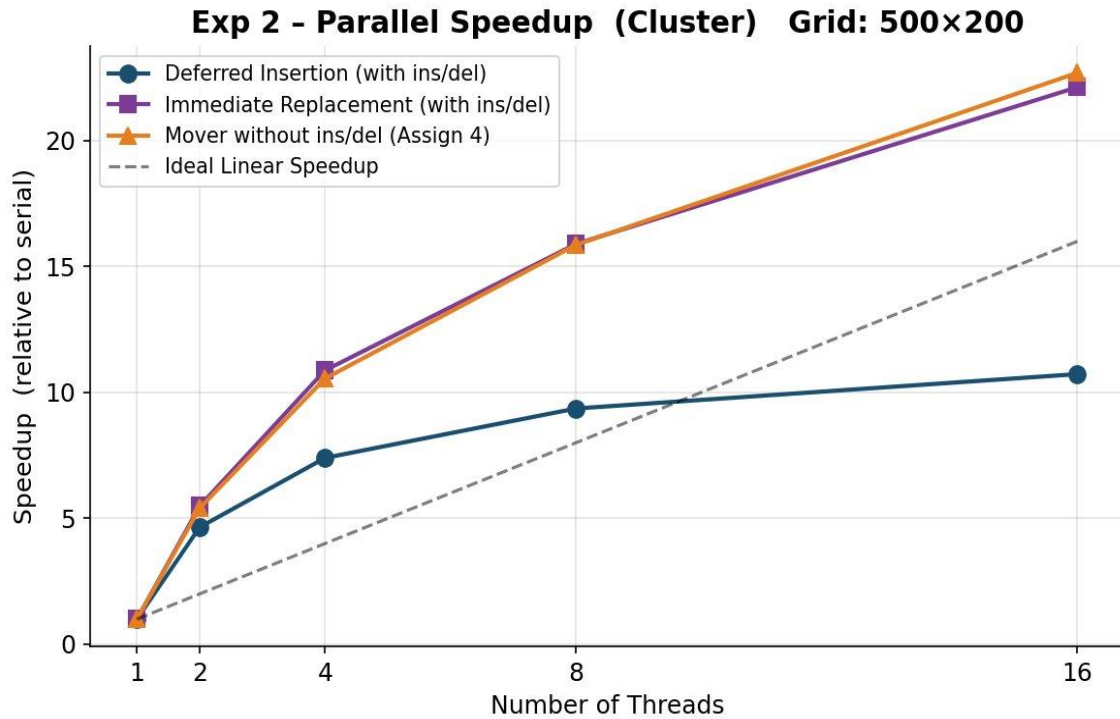


Figure 11: Parallel Speedup (HPC Cluster), Grid 500×200. Both strategies with ins/del achieve super-linear speedup, converging toward the Mover-only baseline.

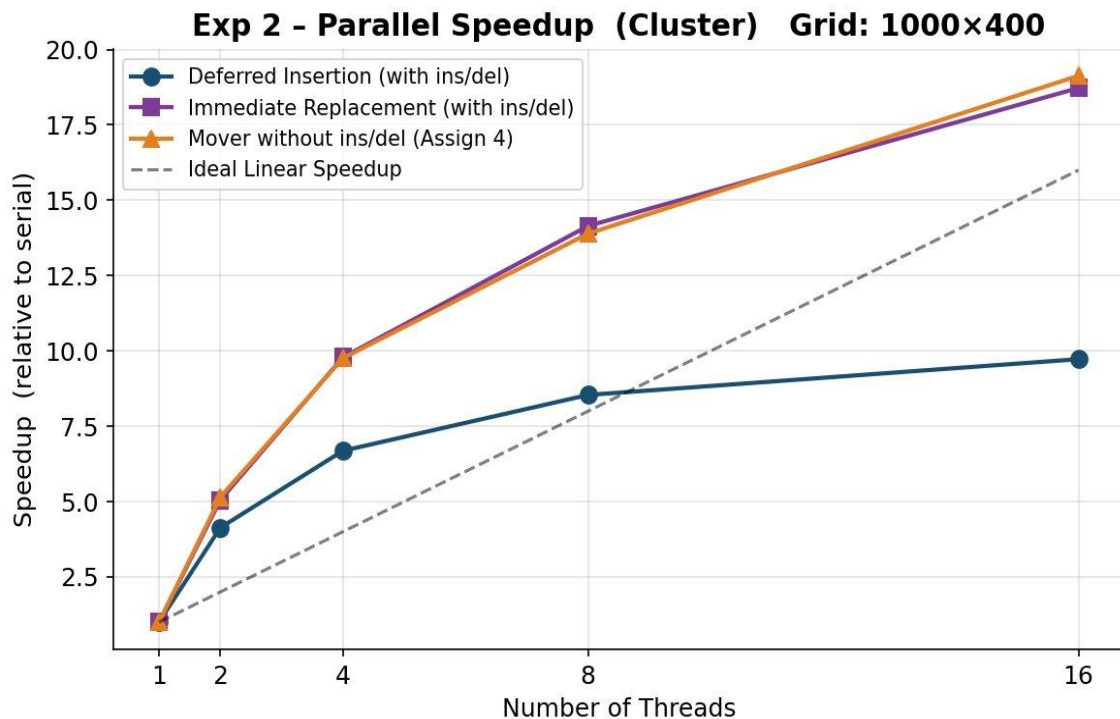


Figure 12: Parallel Speedup (HPC Cluster), Grid 1000×400. Immediate Replacement and Mover-only nearly coincide (~18–19× at 16 threads), showing that ins/del overhead becomes negligible for large grids.

4.1 Observations

- Immediate Replacement consistently outperforms Deferred Insertion in parallel scaling. At 250×100 with 16 threads: ~8.9× vs ~4.8×. The bottleneck in Deferred Insertion is the post-loop serial merge of thread-private deletion lists and the subsequent insertion pass.
- At larger grids (500×200, 1000×400), both strategies converge toward the Mover-without-ins/del baseline, confirming that ins/del overhead diminishes as particle-movement work dominates.
- Super-linear speedup (>16× at 16 threads for 500×200) reflects cache-size effects: each thread's portion of 14M particles fits in per-core private cache, reducing DRAM access rates below the serial case.
- Parallel scalability improvements employed: (1) thread-private deletion counters to eliminate atomic contention; (2) per-thread seeded random number generators (`rand_r`) to avoid global lock contention; (3) preferring Immediate Replacement for production runs due to its lower synchronisation cost.
- Difficulty with Deferred Insertion: the merge of deleted indices after the parallel region is serial, and its cost grows with the fraction of deleted particles per iteration. For the small displacement range used ($\pm\Delta x$, $\pm\Delta y$), this fraction is small but still measurable at high thread counts where the per-thread mover work is tiny.

5 Q3: Particle Distribution Verification

To confirm that insertion logic does not introduce spatial bias, 200k particles were initialised and their positions examined via scatter plot and coordinate histograms.

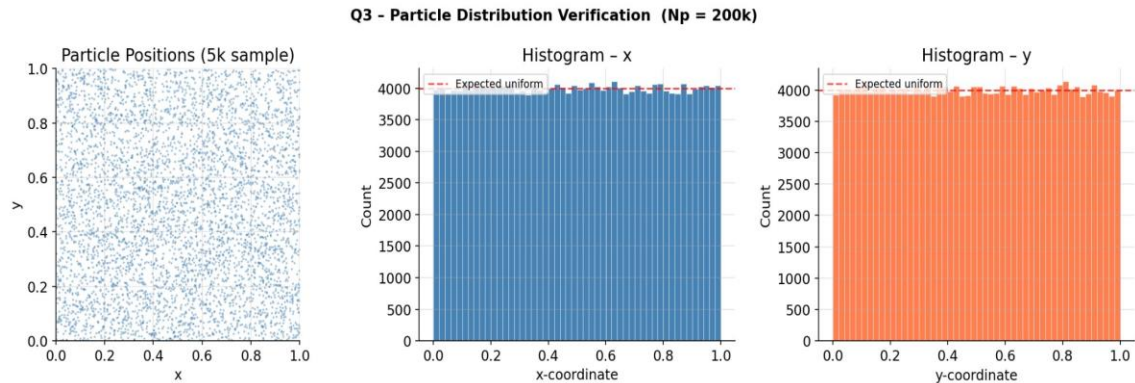


Figure 13: Q3 — Particle Distribution (Np = 200k). Left: scatter of 5k sample; Centre/Right: x and y histograms. All bins track closely to the expected uniform count (red dashed line).

The scatter plot shows no visible clusters or voids. Both histograms are flat and consistent with a uniform distribution — minor fluctuations are within expected statistical noise. This confirms that the random insertion logic correctly places new particles uniformly across the domain without artificial spatial bias.

6 Q4: Per-Particle Execution Time vs. Particles-per-Cell (PPC)

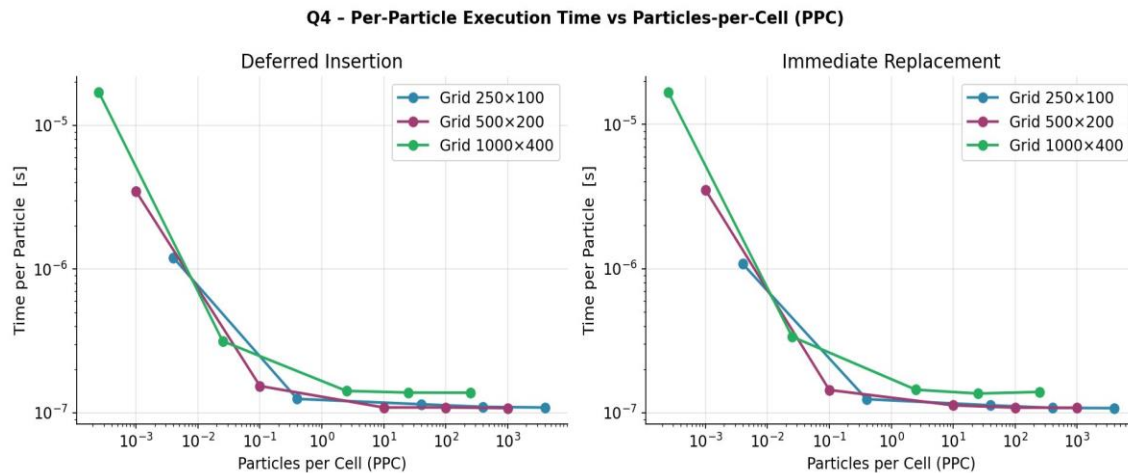


Figure 14: Q4 — Per-Particle Execution Time vs. PPC (Lab PC). Left: Deferred Insertion; Right: Immediate Replacement. Both show identical two-regime behaviour.

6.1 Analysis of PPC Trend

$PPC = N / (N_x \times N_y)$. Two distinct regimes are observed:

- Low PPC (< 0.1): Per-particle time is high and steeply decreasing. The grid mesh must be allocated and zeroed regardless of N , so its fixed cost is amortised over very few particles at low PPC, driving up the per-particle cost. Larger grids (1000×400) exhibit the highest per-particle cost at low PPC because their grid arrays are largest.
- High PPC (> 1): Per-particle time flattens to $\sim 10^{-7}$ s/particle for all grid sizes. The workload is purely particle-driven; each particle contributes a fixed arithmetic cost. All three grid configurations converge to the same floor, confirming that per-particle cost is grid-independent at sufficient particle density.
- Transition ($PPC \approx 0.01\text{--}0.1$): marks the crossover from grid-overhead-dominated to particle-dominated regime. Its position shifts right with increasing grid size — a larger grid requires more particles before particle-work dominates.
- Both Deferred and Immediate strategies produce nearly identical PPC curves, confirming that insertion strategy does not materially affect per-particle cost.

7 Discussion

7.1 Lab PC vs. HPC Cluster

- Clock frequency and IPC: consumer CPUs run at higher boost frequencies with wider out-of-order execution, favouring the sequential particle loop.
- Cache hierarchy: the Lab PC's L3 cache is large enough to hold more of the particle arrays, reducing DRAM traffic for medium-sized problems.
- NUMA effects: cluster nodes use multi-channel NUMA topologies; without thread pinning, cross-NUMA memory accesses inflate latency.
- Cluster timing coarseness: at small N, timer resolution on the cluster is insufficient, slightly inflating measured times.

7.2 Impact of Ins/Del on Scalability

The two main scalability challenges introduced by particle insertion/deletion are:

- Synchronisation overhead (Deferred Insertion): thread-private deletion lists must be merged in a serial post-loop step, creating a barrier that limits parallel efficiency.
- Random number contention: a shared `rand()` is not thread-safe. Per-thread `rand_r()` or PCG generators eliminate this bottleneck.

Immediate Replacement avoids the first issue entirely — since each thread replaces particles within its own partition, no cross-thread communication is needed for slot assignment.

7.3 Super-linear Speedup Explanation

At 16 threads, each thread's working set (~ 7 MB for $14\text{M}/16 = 875\text{k}$ particles $\times 2$ doubles) may fit in per-core L2/L3, whereas the full 112 MB working set of the serial run overflows all cache levels. The resulting reduction in cache miss rate produces throughput gains beyond the theoretical linear ceiling.

8 Conclusion

- Both Deferred Insertion and Immediate Replacement exhibit $O(N)$ serial scaling across all three grid configurations and both platforms.
- The Lab PC outperforms the HPC cluster in serial mode by $\sim 2\text{--}3\times$ at large N , consistent with higher clock frequency, cache effectiveness, and lower memory latency.
- Immediate Replacement parallelises more efficiently than Deferred Insertion, achieving $\sim 8.9\times$ at 16 threads (250×100) to $\sim 19\times$ (1000×400) — matching the Mover-without-ins/del baseline at large grid sizes.
- Particle distribution verification confirms that insertion logic generates uniformly distributed particles with no spatial clustering or bias.
- Per-particle time analysis reveals two regimes: grid-overhead-dominated at low PPC, and a particle-driven plateau at $\sim 10^{-7}$ s/particle at high PPC, independent of grid size.
- Key measures to improve scalability: per-thread random generators, thread-private deletion counters, and preferring Immediate Replacement for production parallel runs.